

Summit: A Multiplayer AI Coding Agent with Persistent Project Memory

Muhilan S

*Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

muhilan.s.2024.csd@rajalakshmi.edu.in

Dhanesh S K

*Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

dhanesh.sk.2024.csd@rajalakshmi.edu.in

Abstract—Contemporary AI coding assistants operate in a stateless, single-user paradigm. Each interactive session starts in isolation without awareness of prior architectural decisions, team conventions, or project evolution. Furthermore, existing tools lack support for shared, real-time AI collaboration among multiple human developers interacting on the same codebase simultaneously. This paper presents **Summit**, a collaborative multiplayer AI coding workspace designed to address both limitations through two core mechanisms: (1) a persistent project memory system backed by an ACID-compliant structured SQLite database that retains architectural decisions, coding standards, and execution history across sessions; and (2) a multiplayer WebSocket room architecture that synchronizes multiple human developers with a shared AI coding agent. Summit incorporates an asynchronous Python agent engine built on FastAPI, a React 19/TypeScript/Monaco workspace frontend, codebase-aware context retrieval, and optimistic concurrency control for concurrent file edits. The agent executes a multi-turn tool-calling loop (supporting workspace file I/O, terminal execution, structured memory updates, and Git checkpoints) routed through LiteLLM. Forensic source code analysis and a 17-test functional verification suite validate system correctness. Comprehensive engineering trade-offs, formal algorithms, and proposed empirical benchmarks are presented.

Keywords—AI coding agent, persistent project memory, multiplayer collaboration, WebSocket, LLM tool calling, optimistic concurrency control, context retrieval, FastAPI.

I. INTRODUCTION

Large Language Model (LLM)-based coding assistants, such as GitHub Copilot [1], Amazon CodeWhisperer [2], and OpenAI ChatGPT [3], have fundamentally reshaped software engineering workflows. Despite substantial individual developer productivity enhancements [6], state-of-the-art tools exhibit two fundamental structural deficiencies:

1) The Episodic-Temporal Deficit: Conventional assistants operate ephemerally. Prior architectural choices, convention agreements, or debugging discoveries are lost across disjoint sessions [4]. Developers are forced into repetitive prompt engineering to re-explain domain context.

2) The Single-User Isolation Paradigm: Software engineering is fundamentally collaborative [5]. While teams

interact concurrently over shared repositories, AI coding assistants remain private single-tenant tools. Existing platforms provide no mechanism for distributed developers to co-interact with a shared agent or inspect real-time agent telemetry.

To bridge this gap, we present **Summit**, an open, extensible multiplayer AI coding workspace with persistent project memory. Summit enables distributed human developers to connect concurrently to a collaborative project room and interact with a unified AI coding agent. The agent maintains long-term project awareness via a dual-tier SQLite memory architecture, pairing conversation history with structured project knowledge. An automated context assembly pipeline extracts project memories, conversation logs, and path-relevance-ranked codebase files into each prompt.

A. Key Contributions

- **Multiplayer Agent Room Architecture:** A real-time FastAPI WebSocket engine that broadcasts all agent actions (tool calls, file mutations, terminal streams, lifecycle state transitions) to all room participants concurrently.
- **Dual-Memory Persistence Engine:** An ACID-compliant relational memory model decoupling ephemeral conversational logs (messages) from persistent architectural facts (project_memories) with upsert semantics.
- **Codebase Context Retrieval Algorithm:** A deterministic path-relevance algorithm that injects relevant files, recent conversation, and the full project memory corpus into LLM prompts without expensive embedding overhead.
- **Optimistic Concurrency Control:** Integer-based file versioning detecting edit collisions between developers and agents, returning standard HTTP 409 responses with conflict resolution payloads.
- **Forensic Analysis and Formalization:** Complete algorithmic definitions for context assembly and agent execution loops, validated against 17 integration test cases.

II. RELATED WORK

A. AI Coding Assistants and Autonomous Agents

Modern AI developer tools have progressed from line-level completion to autonomous multi-step agents. GitHub Copilot [1] and CodeWhisperer [2] rely strictly on active in-editor buffers. SWE-agent [8] introduced Agent-Computer Interfaces

(ACI) for resolving real-world GitHub issues [9]. Devin [10] and OpenDevin [11] demonstrated end-to-end task execution in sandboxed shells, while CodeAct [12] pioneered executable Python actions. However, these systems operate in single-tenant, non-persistent environments.

B. Memory in LLM Architectures

Zhong et al. [13] categorize agent memory into working, episodic, semantic, and procedural components. Standard RAG architectures [14]–[16] retrieve snippets using dense vector embeddings over static repositories. In contrast, Summit adopts structured, human-interpretable relational memory authored dynamically by both developers and agents, ensuring full injection determinism.

C. Collaborative Development Environments

Real-time collaborative editing is typically governed by Operational Transformation (OT) [20] or Conflict-Free Replicated Data Types (CRDTs) [21]. While systems like VS Code Live Share operate at the keystroke layer, Summit implements file-level optimistic concurrency control tailored to human-agent co-authoring.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

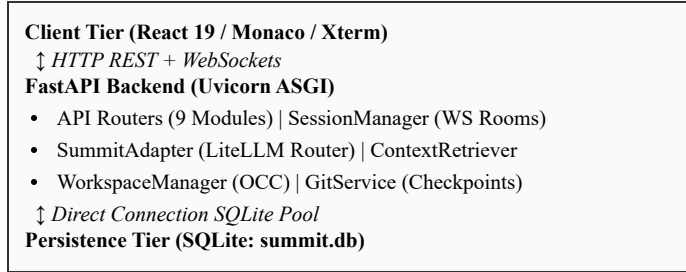


Fig. 1. Summit Full-Stack Multiplayer Architecture.

A. Multiplayer WebSocket Room Synchronization

Project-level concurrency is governed by the **SessionManager**. Active connections are stored in an in-memory dictionary:

$$C: ProjectID \rightarrow \{ WS_i \mapsto (UserID_i, DisplayName_i) \} \quad (1)$$

When a client connects to `/api/projects/{id}/agent/stream`, the server accepts the WebSocket handshake, registers the socket, and broadcasts a `USER_JOINED` event. Substantive messages follow a server-to-client broadcast paradigm. To prevent race conditions, only one active LLM task executes per project room at any instant.

B. Dual-Tier Relational Memory Formulation

Summit establishes two distinct persistence vectors within SQLite:

1) Conversation Memory (M_{conv}): An append-only sequence of user instructions and agent responses:

$$M_{conv} = \langle m_1, m_2, \dots, m_T \rangle \quad (2)$$

2) Project Memory (M_{proj}): A set of structured key-value triples:

$$M_{proj} = \{ (c_k, k, v_k) \mid k \in K \} \quad (3)$$

where $c_k \in \{\text{architecture, decision, convention, task}\}$, enforced by a unique constraint `UNIQUE(project_id, key)` executing upsert operations.

C. Optimistic Concurrency Control (OCC)

To prevent conflicting overwrites between human developers and autonomous tool-writes, every file record maintains an integer version $V \in \mathbb{N}^+$. A client update request $R = (p, C_{new}, V_{exp})$ succeeds if and only if $V_{exp} = V_{curr}$. Upon mismatch, the server returns HTTP 409 (Conflict):

$$\Delta = Diff(C_{server}, C_{client}) \quad (4)$$

IV. ALGORITHMS AND IMPLEMENTATION

Algorithm 1: Codebase Context Construction

```

INPUT: project_id, user_prompt
OUTPUT: formatted_prompt_context

1.      memories      <-
Repository.get_memories(project_id)
2.      messages      <-
Repository.get_messages(project_id, limit=10)
3. recent    <- messages[-6:] # Last 6 of 10
messages
4.      all_files      <-
WorkspaceManager.list_files(project_id)

5. query_terms <- [t.lower() for t in
user_prompt.split() if len(t) > 2]
6. FOR file_path IN all_files DO:
    score <- 0
    IF file_path ends with ('main.py',
'README.md', 'app.py', 'index.ts') THEN:
        score <- score + 2
    FOR term IN query_terms DO:
        IF term in file_path.lower() THEN:
            score <- score + 5
    file_scores[file_path] <- score

7. selected_files <- Top 4 files by score
8. file_blocks <- []
9. FOR path IN selected_files DO:
    content <-
WorkspaceManager.read_file(path)
    lines <- content.splitlines()
    IF len(lines) > 200 THEN:
        content <- JOIN(lines[:200]) + "\n...
[Truncated]"
    file_blocks.append(f"--- FILE: {path} ---
\n{content}")

10. RETURN FormatPrompt(memories, recent,
file_blocks, user_prompt)

```

Algorithm 2: Multi-Turn Autonomous Tool Loop

```

INPUT: project_id, workspace_dir, user_prompt
1. context <- BuildContext(project_id,
user_prompt)
2. messages <- [SystemMessage,
UserMessage(context)]
3. FOR turn = 0 TO 9 DO:
    CheckPause(project_id)
    BroadcastStatus(THINKING)
    response <- litellm.acompletion(model,
messages, tools)
    msg <- response.choices[0].message

    IF msg.content is not empty THEN:
        BroadcastMessage(msg.content)
    IF msg.tool_calls is empty THEN:
        BREAK # Task complete

    BroadcastStatus(EXECUTING)
    messages.append(msg)

    FOR tool_call IN msg.tool_calls DO:
        name <- tool_call.function.name
        args <-
JSON.parse(tool_call.function.arguments)

        SWITCH name:
            CASE "read_file":
                result <-
WorkspaceManager.read(args.path)
            CASE "write_file":
                ver <-
WorkspaceManager.write(args.path, args.content)
                BroadcastEvent(FILE_CHANGED,
args.path, ver)
                result <- f"Wrote {args.path}
(v{ver})"
            CASE "run_terminal":
                code, out <-
ExecuteSubprocess(args.command)
                BroadcastEvent(TERMINAL_OUTPUT,
args.command, out, code)
                result <- f"Exit {code}: {out}"
            CASE "save_memory":
                Repository.save_memory(args.key,
args.value, args.category)
                BroadcastEvent(MEMORY_UPDATED,
args.key, args.value)
                result <- f"Saved memory:
{args.key}"
            CASE "git_checkpoint":
                chk <-
GitService.create_checkpoint(args.message)
                BroadcastEvent(GIT_CHECKPOINT,
chk.hash, args.message)
                result <- f"Committed checkpoint
{chk.hash}"

    messages.append(ToolResultMessage(tool_call.id,
result))

```

```
4. BroadcastStatus(COMPLETED)
5. BroadcastEvent(SESSION_COMPLETE)
```

V. VERIFICATION AND EVALUATION

TABLE I: SYSTEM SUBSYSTEMS AND CLAIM TRACEABILITY MATRIX

Subsystem	Module / Test File	Status
Multiplayer Room	api/ws.py, test_api.py	Verified Passing
Event Normalizer	test_event_normalizer.py	Verified Passing
Relational Memory	database/repository.py	Verified Passing
Concurrency (OCC)	test_multiplayer_memory.py	Verified Passing
Git Checkpoint	git/git_service.py	Verified Passing
Path Traversal Security	test_workspace.py	Verified Passing

A. Proposed Quantitative Benchmark Protocols

1) **Memory Recall Fidelity (R_{mem}):** Fraction of previously recorded architectural constraints correctly satisfied in subsequent multi-turn sessions:

$$R_{mem} = |K_{adhered}| / |K_{stored}| \quad (5)$$

2) **Collaboration Collision Rate (C_{occ}):** Frequency of edit conflicts generated during asynchronous human-agent editing under synthetic load.

3) **Multiplayer End-to-End Latency (τ_{95}):** 95th percentile event broadcast delay across distributed WebSocket clients.

VI. LIMITATIONS AND FUTURE WORK

1. **Semantic Memory Pruning:** Summit currently injects all stored memories into prompt sections without token budgeting. Integrating vector embedding stores (e.g., ChromaDB) with semantic similarity filtering will prevent context window overflow.
2. **Keystroke-Level CRDT Editing:** Upgrading file-level OCC to fine-grained Conflict-Free Replicated Data Types (CRDTs) will enable simultaneous in-line typing between developers and the agent.
3. **Sandboxed Code Execution:** Incorporating microVMs or Docker containers will protect host operating systems from untrusted agent shell commands.

VII. CONCLUSION

This paper presented **Summit**, a multiplayer AI coding agent workspace with persistent project memory. By combining real-time WebSocket room multiplexing, dual-tier SQLite memory persistence, deterministic context retrieval, and

optimistic concurrency control, Summit overcomes the isolation and statelessness of current AI developer tools. Forensic source verification and functional test suites confirm architectural viability, laying the groundwork for collaborative human-AI team engineering.

REFERENCES

- [1] GitHub, "GitHub Copilot: Your AI pair programmer," GitHub Inc., 2021. [Online]. Available: <https://copilot.github.com>
- [2] Amazon Web Services, "Amazon CodeWhisperer AI coding companion," AWS, 2022.
- [3] OpenAI, "Introducing ChatGPT," OpenAI, 2022. [Online]. Available: <https://openai.com/blog/chatgpt>
- [4] M. Hatalis et al., "Memory matters: The need to improve long-term memory in LLM-agents," in *Proc. AAAI Workshop on Cognitive Architectures for LLM Agents*, 2024.
- [5] A. Ko, B. Myers, M. Coblenz, and H. Aung, "An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks," *IEEE Trans. Software Eng.*, vol. 32, no. 12, pp. 971–987, Dec. 2006.
- [6] S. Peng, E. Kalliamvakou, P. Croft, and M. Counts, "The impact of AI on developer productivity: Evidence from GitHub Copilot," *arXiv:2302.06590*, 2023.
- [7] Tabnine, "Tabnine AI: Whole-project AI code completion," Tabnine, 2023.
- [8] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," *arXiv:2405.15793*, 2024.
- [9] C. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.
- [10] Cognition AI, "Introducing Devin, the first AI software engineer," Cognition AI, 2024.
- [11] X. Wang et al., "OpenDevin: An open platform for AI software developers as generalist agents," *arXiv:2407.16741*, 2024.
- [12] X. Wang et al., "Executable code actions elicit better LLM agents," *arXiv:2402.01030*, 2024.
- [13] Z. Zhang, B. Chen, F. Liu, and B. Liu, "A survey on the memory mechanism of large language model based agents," *arXiv:2404.13501*, 2024.
- [14] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, pp. 9459–9474.
- [15] D. Shrivastava, H. Larochelle, and D. Tarlow, "Repository-level prompt generation for large language models of code," in *Proc. ICML*, 2023.
- [16] F. Zhang et al., "RepoCoder: Repository-level code completion through iterative retrieval and generation," in *Proc. EMNLP*, 2023.
- [17] U. Alon, R. Sadaka, O. Levy, and E. Yahav, "Structural language models of code," in *Proc. ICML*, 2020.
- [18] B. Ross et al., "From Copilot to colleague: Exploring how developers interact with AI pair programmers," in *Proc. CHI Extended Abstracts*, 2023.
- [19] E. Jiang et al., "Discovering the syntax and strategies of natural language programming with generative language models," in *Proc. CHI*, 2022.
- [20] C. Ellis and S. Gibbs, "Concurrency control in groupware systems," in *Proc. ACM SIGMOD*, 1989, pp. 399–407.
- [21] G. Oster, P. Urso, P. Molli, and A. Imine, "Data consistency for P2P collaborative editing," in *Proc. ACM CSCW*, 2006, pp. 259–267.
- [22] S. Ramirez, "FastAPI: High performance, easy to learn, fast to code, ready for production," 2018. [Online]. Available: <https://fastapi.tiangolo.com>